

MESSENGERBOT R // CODE ON A DIET

카카오톡 봇으로 배우는 코드 다이어트 6단계

복잡한 코드를 짧게가 아니라 '안 읽어도 되게' 만드는 법

Rhino / ES5

리팩터링

초보자용

복잡함은 '길어서'가 아니라 '섞여서' 생긴다

코드 다이어트 = 살 빼기가 아니다. 짧게 우겨넣은 코드는 오히려 안 읽힌다.

진짜 목표는

목표는 '짧은 코드'가 아니라 '안 읽어도 되는 코드' 다.

1

함수 하나가 여러 일을 함

검증·저장·응답을 한 함수에 다 넣으면 머릿속이 터진다

2

if 가 계단처럼 중첩됨

괄호가 깊어질수록 눈이 오른쪽으로 밀려난다

3

뜻 모를 값이 박혀 있음

86400000 이 뭔지 아무도 모른다

4

같은 패턴이 복사·붙여넣기

명령어마다 if-else 를 새로 쓴다

함수를 잘게 쪼개라

함수 하나 = 한 가지 일. 안에 // 검증 // 저장 처럼 구간을 나누고 있다면, 그 구간이 곧 함수다.

❌ Before - 한 함수가 다 함

```
function onNewbie(msg) {  
  var n = msg.content.split(" ");  
  if (n.length !== 4) {  
    msg.reply("양식 틀림"); return;  
  }  
  if (n[0] !== "남" && n[0] !== "여") {  
    msg.reply("성별 틀림"); return;  
  }  
  // 나이·지역 검증 // DB 저장 // 안내  
} // 60줄 넘어감
```

✅ After - 흐름만 남김

```
function onNewbie(msg) {  
  var f = parseNick(msg.content);  
  if (!isValid(f)) {  
    reject(msg); return;  
  }  
  save(f);  
  goNext(msg);  
}  
  
// 검증·저장·안내는 각자 함수로.  
// onNewbie 는 '순서'만 말함
```

이름만 읽어도 뭘 하는지 보인다 → 안을 안 들여다봐도 됨

중첩 if를 '조기 반환'으로 편다

아닌 경우를 먼저 걸어내면 본체가 평평해진다. 계단 → 일자.

❌ 화살표 모양

```
if (room) {  
  if (isOn(room)) {  
    if (isMember(user)) {  
      if (!isMuted(user)) {  
        doReply();  
      }  
    }  
  }  
}
```

// 눈이 계속 →→→

✅ 위에서 아래로

```
if (!room) return;  
if (!isOn(room)) return;  
if (!isMember(user)) return;  
if (isMuted(user)) return;
```

doReply();

// 통과 못 하면 위에서 끝.

// 본체는 한 줄, 안 밀림

붓 코드에서 특히 유용 — '이 방/이 사람은 무시' 조건을 위에 쪽 나열

STEP 3

뜻 모를 값에 이름을 붙여라

코드 중간에 박힌 숫자·문자는 '이게 뭐지?'를 만든다. 이름이 곧 설명이다.

❌ 매직 넘버

```
if (now - last > 604800000)

    kick(user);

setTimeout(f, 300000);

if (age < 20) ...

// 604800000? 300000?

// 매번 계산해야 함
```

✅ 이름 있는 상수

```
var WEEK_MS = 7*24*60*60*1000;

var CACHE_MS = 5*60*1000;

var MIN_AGE = 20;

if (now-last > WEEK_MS)

    kick(user);

if (age < MIN_AGE) ...
```

보너스 맨 위에 상수를 모아두면 '설정 패널'이 된다. 나이 제한·캐시 시간·마감일을 코드 안 뒤져도 한곳에서 바꿈.

긴 if-else 를 '데이터'로 바꿔라

분기가 데이터가 되면 로직이 사라진다. 명령어 추가도 한 줄.

❌ 명령어마다 if

```
if (cmd=="가위") r="...";  
else if (cmd=="바위") r="...";  
else if (cmd=="보") r="...";  
else if (cmd=="도움말") r="...";  
else if (cmd=="공지") r="...";  
else if (cmd=="규칙") r="...";  
// 명령어 늘 때마다 if 가 붙음
```

✅ 객체 하나로

```
var CMD = {  
  "가위": rps, "바위": rps,  
  "보": rps,  
  "도움말": help, "공지": notice  
};  
  
var fn = CMD[cmd];  
  
if (fn) fn(msg);    // 추가 = 한 줄
```

if-else 20줄 → 표 한 개. 봇 명령어가 많을수록 차이가 폭발한다

STEP 5

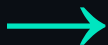
한 파일이 500줄 넘으면 나눠라

기능 단위로 쪼개면 '어디 고치지?' 할 때 바로 찾아간다.

✗ Before

bot.js

온보딩
메뉴검색
방관리
멤버명단
게임
출력문구
... 1500줄



✓ After - 기능 단위

main.js

흐름만 — 어떤 순서로 도는지

onboarding.js

신입 상태머신

menu.js

맛집 검색

members.js

멤버 명단·통계

messages.js

출력 문구 모음

메뉴 기능 고칠 땐 *menu.js* 만 열면 끝. 스크롤 지옥 탈출.

좋은 이름은 주석을 없앤다

주석을 자꾸 달게 된다면, 그건 이름이 제 역할을 못 한다는 신호다.

✗ 주석 없으면 모름

```
var d = getD(u);  
  
// 마지막 활동 후 지난 일수  
  
var r = chk(n);  
  
// 닉 형식 맞는지  
  
if (t) send(m);
```

✓ 이름이 곧 설명

```
var daysSinceActive =  
    getDaysSinceActive(user);  
  
var isValidNick =  
    checkNickFormat(nick);  
  
if (isNewbie) sendGuide(msg);
```

규칙 불리언은 is/has 로 (isMuted, hasJoined) · 함수는 동사로 (sendGuide, parseNick) · 축약 금지 (u→user, msg는 관용이라 OK)

다이어트할 때 하지 말 것

"짧게"에 집착하지 마라

```
// 짧지만 해독 필요  
var r=a.filter(x=>x.g=="남")  
    .map(x=>x.n).join(",");
```

```
// 길어도 명확  
var males = members.filter(isMale);  
var names = males.map(getName);  
var line = names.join(", ");
```

- ⊗ 한 줄에 다 우겨넣기 (읽는 데 더 오래 걸림)
- ⊗ 한 번에 전부 리팩터링 (깨지면 원인 못 찾음)
- ⊗ 돌아가는 코드를 이유 없이 갈아엎기

복잡한 코드 앞에서 막막하면, 이 순서로

1

긴 함수부터 쪼갬다

효과가 제일 크다. 여기서 대부분 해결

2

중첩 if 를 얼리 리턴으로 편다

계단을 일자로

3

긴 if-else 를 객체로 바꾼다

봇 명령어 처리에 직방

4

안 읽히는 변수 이름을 고친다

주석이 사라진다

5

그래도 크면 파일을 나눈다

기능 단위로

핵심 한 번에 다 하지 말고 하나씩. 조금 고치고 테스트, 조금 고치고 테스트.